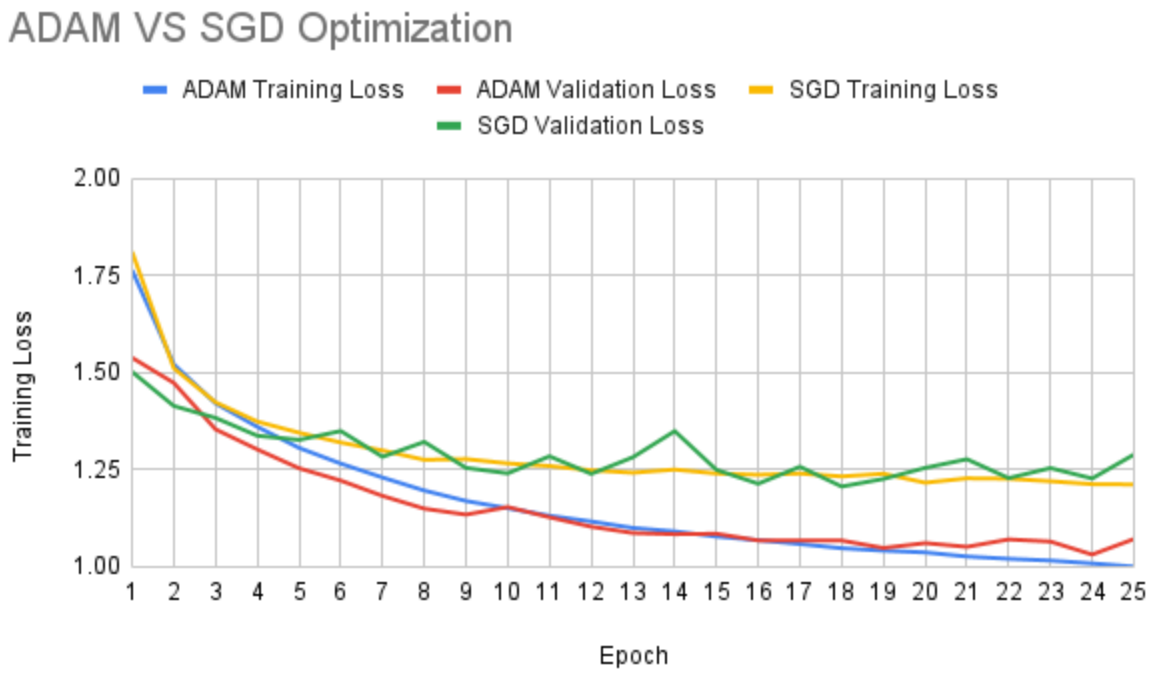
**Model Architecture and Training Setup**

The model used for this assignment is of CNN architecture utilizing the CIFAR-10 dataset and PyTorch for multipurpose model training. The model itself loads in 20 images from the sample provided and then trains the model using those images.

The model itself includes three convolutional layers, one pooling layer, two linear layers, and one theoretical dropout layer which went unused. All layers besides the last linear layer were passed with ReLU activation and the layers are flattened in between the convolutional layers and the fully connected linear layers. The model is then created and if NVIDIA CUDA is available it is moved to CUDA for faster training setup. The kernel size for convolutional layers are 3x3 or 9 and the linear layers start with a in feature size of 64 and out of 32 and ends with a in feature size of 32 and out feature size of 10.

For training both SGD and ADAM optimization was tested and compared (see chart below) with a total epoch count of 25. For SGD optimization a learning rate of 0.005 and a momentum value of 0.9 to help get past any local minimum values. For ADAM optimization a learning rate of 0.001 was used. 25 epoch count was used to help determine the relative stopping point for training in the future. That “Goldilocks Zone” is different for each optimizer with the SGD optimizer being around roughly 7-10 epochs and the ADAM optimizer being around 14 epochs before the model potentially starts overfitting.




**ADAM VS SGD Optimization Results**

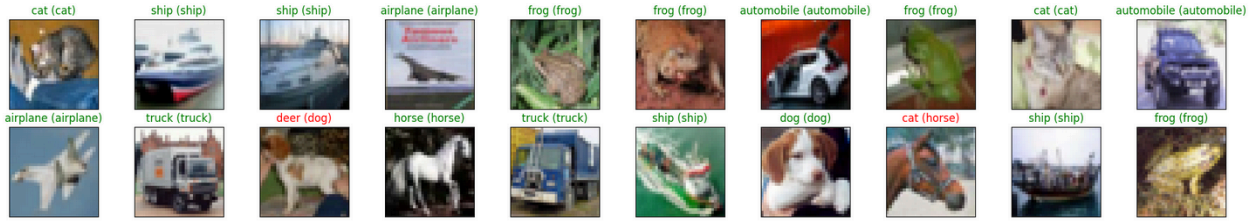
Overall, the training was fairly successful and even though 25 epochs is a bit much for this small of a dataset, it helps define when we should cut off training in the future. Doing so, should provide better yield for less epochs and time spent.

**Training Results**

The results were mixed overall, with an accuracy of 64 percent under ADAM optimization, and 58 percent accuracy under SGD optimization. Unfortunately, this is below the expected minimum; however, there are a few big reasons for this result. One of the reasons was data augmentation not being used within the sampleset. Using data augmentation would allow the model to train with the same pictures, but slightly altered usually resulting in different training data that is more diverse. Another reason for the poor results could be the possible overfitting of this model during training which is less likely since the results were just tested against the training data, but could become an issue when given an image not part of the training dataset.

The last possible reason that comes to mind, is the parameters of the model need more fine-tuning in order to give better results. Multiple models were tested, and the best parameters were chosen, but there most likely are better parameters that would be hard to determine without utilizing separate tools or math to gather.

Although the SGD optimization method did prove to be mostly a failure, the ADAM optimized model did prove to be at least within the ballpark of successfully identifying most images. Both models had trouble figuring out if the image was a cat, bird, or dog; however, it was fairly accurate with ships, trucks, and automobiles. Here is a test result visualization from the ADAM optimized model.

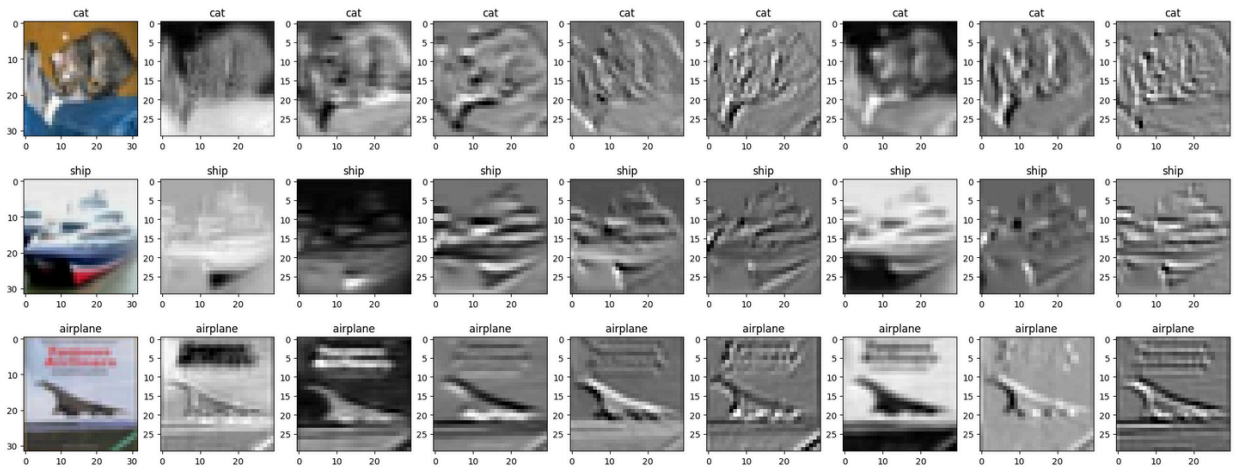



**ADAM Optimization Results**

Overall, The results are definitely disappointing and if more time was spent on training and fine-tuning the model the results probably would have been up to standard. More epochs could have been added to meet that threshold; however, that would just result in overfitting and make the model way too hyperactive on the specific dataset leading to errors from unique pictures.

**Feature Map Visualization**

A part of the data training is feature maps extracting data from the images that are sent during the training stages. To visualize this a hook was registered to grab the data while the model was being evaluated and those “activated” images are then extracted and visualized. In the image below you will see 27 images which contain three base images from different classes and 8 feature maps that the model uses to train.




**Feature Map Visualization Amongst 3 Classes**

The images are chosen as the first three images that aren't the same class in a very hardcoded way, but since we know the dataset is static we can do it this way. Overall, these feature maps give a glimpse at the model using different transpositions of the data to get more data from it.

**Maximally Activating Images**

It can be a good idea to check which images are causing high activation within your filters specified. You can also get a vibe on how the actual model views the test images you are providing it. Here are three filters (1, 2, 4) from the second convolutional layer in the model and their corresponding images.

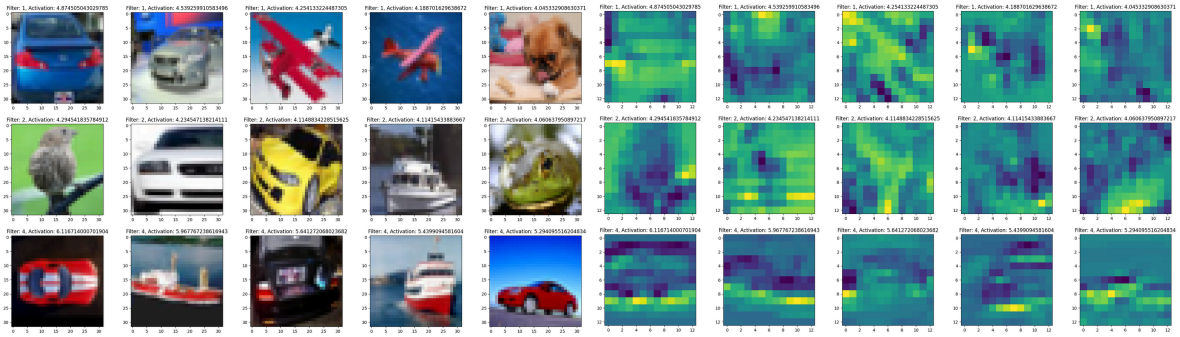


**Image Filter Activations**

There are some patterns here that provide evidence of why the model was so good at detecting automobiles, but was so terrible at detecting birds. As you can see most automobiles the filter clearly shows that it looks for the headlights on those vehicles, but for the birds it really doesn't know where the bird is and instead looks at the background.

In terms of whether or not each filter appears to respond to general features or class-specific patterns it seems that it is looking for general features as there are patterns, but they don't follow the class specifically all the time. Some of the cars have lights that the filter picked up on, but if the color was black instead of red the filter completely ignored it which makes me believe it is acting on just features alone.

Overall, it is very interesting to look at each filter because it gives a glimpse into how the model itself is classifying items based on what it perceives as details. It also can give you an idea at what patterns your model is bad at identifying.

## Discussion

This assignment was a great way to dive head first into making, training, and testing a full CNN model. It did end up underperforming; however, a lot of the information learned can be used and applied to future models that hopefully will perform a bit better. Personally, I believe that a lot of the problems were caused by either too finely tuning the model or purely just errors on how the model was set-up. In future assignments and models I hope to elevate the accuracy more and this was a great starting point.